

The Complete DevOps Tutorial

The Complete DevOps Tutorial — Concepts, Tools, Roadmap, and Real Incidents

Not a tool-by-tool manual — a field guide to why DevOps exists, how its core toolchain fits together (Git → CI/CD → containers → orchestration → IaC → observability → security), a structured learning roadmap, and documented real-world incidents showing what happens when each piece fails.

Table of Contents

Part I — Foundations 1. [What DevOps actually is \(and isn't\)](#) 2. [Linux & shell fundamentals every DevOps engineer needs](#) 3. [Git in production: branching strategies that actually get used](#)

Part II — CI/CD 4. [CI vs. CD vs. Continuous Deployment](#) 5. [Building a pipeline as code](#) 6. [Artifact management & versioning](#)

Part III — Containers & Orchestration 7. [Docker, deep dive](#) 8. [Kubernetes, deep dive](#) 9. [Helm: templating Kubernetes manifests](#) 10. [GitOps: the cluster as a git repo](#)

Part IV — Infrastructure as Code & Config Management 11. [Terraform, deep dive](#) 12. [Ansible: configuration management](#) 13. [Terraform vs. Ansible vs. Pulumi](#)

Part V — Cloud Fundamentals 14. [AWS core services for DevOps](#)

Part VI — Observability & SRE 15. [The three pillars: metrics, logs, traces](#) 16. [Prometheus + Grafana](#) 17. [Centralized logging: the ELK/EFK stack](#) 18. [Distributed tracing: OpenTelemetry + Jaeger](#) 19. [SRE concepts: SLIs, SLOs, error budgets, postmortems](#)

Part VII — Security 20. [DevSecOps: shifting security left](#)

Part VIII — Deployment Strategies & Resilience 21. [Blue-green, canary, and rolling deployments](#) 22. [Chaos engineering](#) 23. [Disaster recovery: RTO, RPO, and backups](#)

Part IX — Real-World Incidents 24. [Knight Capital: \\$440 million in 45 minutes](#) 25. [AWS S3, February 2017: a typo takes down half the internet](#) 26. [GitLab.com, 2017: the database incident and its radically public postmortem](#) 27. [Meta, October 2021: the BGP outage that locked engineers out of their own building](#) 28. [Netflix: engineering for failure on purpose](#) 29. [Etsy: from twice-a-week deploys to fifty times a day](#)

Part X — The Roadmap 30. [A structured roadmap, 0–12 months](#)

Part XI — Build 31. [Projects to build, in order](#)

Part I — Foundations

What DevOps actually is (and isn't)

DevOps is not a job title, a tool, or “the person who manages our Kubernetes cluster.” It’s a response to a specific, historically real problem: **development teams were measured on shipping change, operations teams were measured on preventing change (uptime), and those two incentives were structurally at war.** A dev team finishes a feature, throws it over a wall to ops, ops is the one paged at 3 AM when it breaks, and neither side has much reason to help the other move faster — this is literally called “the wall of confusion” in the literature that coined the term.

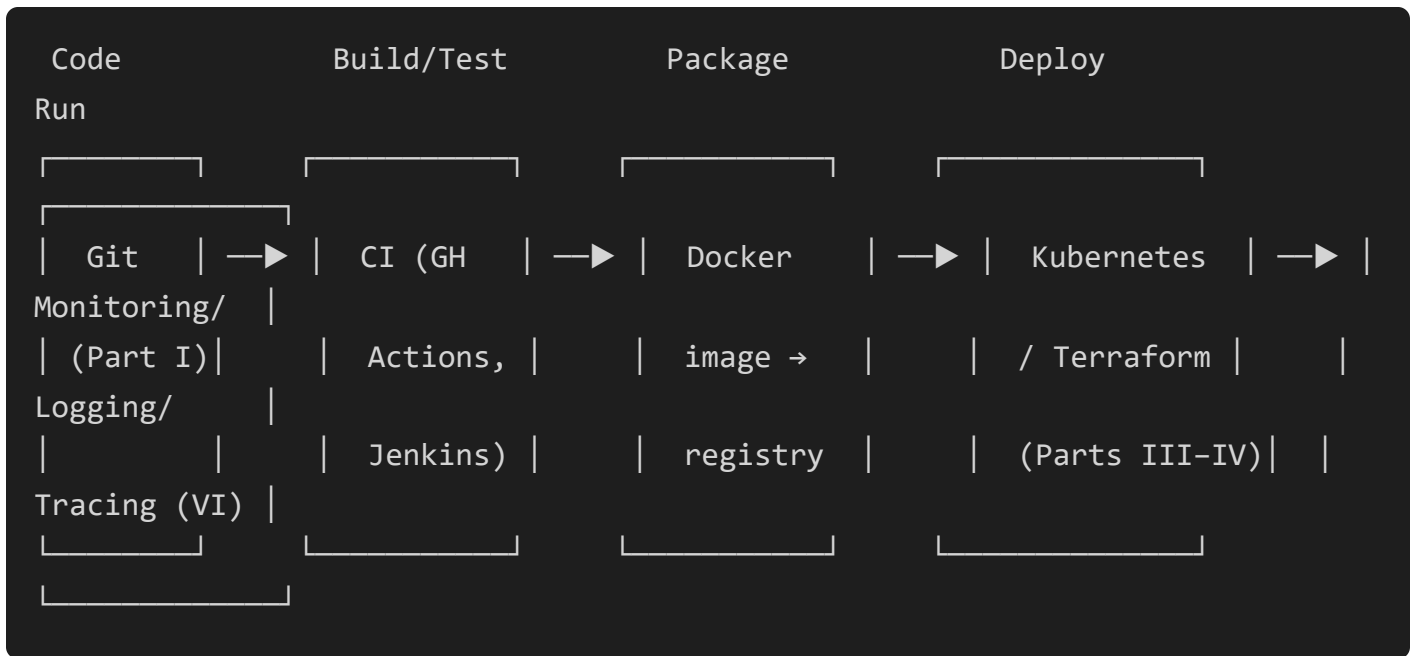
DevOps is the set of practices, culture, and tooling built to collapse that wall: the same people (or tightly collaborating teams) build, ship, run, and are accountable for what they run in production.

CALMS is the standard framework for what “doing DevOps” actually requires, and it’s useful precisely because it’s not tool-shaped:

Letter	Stands for	What it means in practice
C	Culture	Blameless postmortems, shared on-call, no “ops team” that only gets involved after launch
A	Automation	If a human runs the same manual step twice, it should become a script or pipeline stage the third time
L	Lean	Small batch sizes — ship a small change and see its effect, instead of a quarterly “big bang” release
M	Measurement	You cannot improve what you don’t measure — deploy frequency, lead time, change failure rate, time to restore (the four DORA metrics)
S	Sharing	Knowledge, incidents, and tooling are shared across teams, not hoarded

Why this matters before any tool. Handing a team Docker, Kubernetes, and a Jenkins pipeline without changing who’s accountable for production just moves the wall — now it’s between “the team that writes YAML” and everyone else. Every case study in Part IX is, underneath the technical root cause, also a story about a process or culture gap that let the technical failure become a catastrophe instead of a contained incident.

The toolchain map — this guide’s Parts II–VII exist because a working DevOps pipeline is, in practice, always assembled from the same handful of categories:



Every arrow in that diagram is a place a real incident in Part IX happened — this guide follows the diagram left to right, then spends Part IX walking back through it with real failures.

Linux & shell fundamentals every DevOps engineer needs

Nearly everything downstream — Docker, Kubernetes nodes, cloud VMs, CI runners — is Linux. A few categories of command cover most day-to-day debugging:

```

# Process & resource inspection
ps aux | grep node           # is the process even running?
top / htop                   # live CPU/memory usage
df -h                        # disk space (the #1 cause of mystery outages)
free -h                     # memory usage
netstat -tulpn / ss -tulpn  # what's listening on which port

# Logs
tail -f /var/log/app.log     # follow a log file live
journalctl -u myservice -f  # follow a systemd service's logs

# File & permissions
chmod 600 id_rsa             # owner read/write only – required for SSH keys
chown appuser:appgroup /app # change ownership

# Networking
curl -v https://api.internal/health # -v shows headers, redirects, TLS handshake
dig api.internal               # DNS resolution

```

Why this matters. A container orchestrator, a cloud console, and a CI dashboard are all, underneath, abstractions over Linux processes, filesystems, and network sockets. When Kubernetes says a pod is `CrashLoopBackOff`, the actual diagnosis almost always happens with `kubectl logs`, which is fundamentally the same skill as reading `journalctl` output on a bare VM. Skipping this layer to jump straight to Kubernetes is the single most common gap that shows up as “I don’t know why to even start debugging” during an incident.

A short shell script is also usually the first piece of “automation” a DevOps engineer writes — worth being fluent in, not just able to copy-paste:

```
#!/usr/bin/env bash
set -euo pipefail # exit on error, exit on unset variable, fail a pipeline
if any stage fails

APP_DIR="/opt/myapp"
LOG_FILE="/var/log/myapp/deploy.log"

echo "$(date) - starting deploy" >> "$LOG_FILE"
cd "$APP_DIR"
git pull origin main
npm ci --production
systemctl restart myapp
echo "$(date) - deploy finished" >> "$LOG_FILE"
```

set -euo pipefail is the single highest-value line in any production shell script.

Without it, a script silently continues past a failed command (a bad `cd`, a failed `git pull`), and the *next* command runs in the wrong directory or against stale code — a whole category of “the deploy script said success but nothing actually deployed” incidents traces back to missing this line.

Git in production: branching strategies that actually get used

Git itself is a tool everyone knows the basics of; the part that actually differs between teams — and causes real friction — is the **branching strategy**, which is really a decision about how much risk you tolerate in exchange for how much process overhead.

Strategy	How it works	Trade-off
GitFlow	Long-lived <code>develop</code> + <code>main</code> , feature branches, release branches, hotfix branches	Very structured, but slow — merging <code>develop</code> into <code>main</code> for a release is itself a risky, infrequent event. Rare in modern web teams; still common in shops shipping versioned desktop/embedded software.
GitHub Flow	<code>main</code> is always deployable; every change is a short-lived feature branch, PR-reviewed, merged straight to <code>main</code> , deployed immediately	Simple, fast, matches continuous deployment well. Needs strong CI and feature flags to be safe, since there's no "release branch" buffer.
Trunk-Based Development	Everyone commits to <code>main</code> (or very short-lived branches merged within a day), gated by feature flags rather than by branches	What most high-deploy-frequency companies (Google, Meta, Etsy — Part IX) actually run. Requires the most CI/testing discipline of the three, since there's no isolation buffer at all.

Pattern. The DORA research (the same research that produces the four metrics in the CALMS table above) found that trunk-based development with short-lived branches correlates with higher deploy frequency *and* lower change failure rate — the opposite of the intuition that "more branches = more safety." The reason: long-lived branches accumulate large, risky diffs that are hard to review and hard to reason about when they finally merge; short-lived branches force small, reviewable, low-risk changes.

```
# The shape of a trunk-based workflow in practice
git checkout -b add-retry-logic main
# ... small, focused change ...
git push origin add-retry-logic
# open a PR, get one review, merge within hours – not days
git checkout main && git pull
git branch -d add-retry-logic
```

Part II — CI/CD

CI vs. CD vs. Continuous Deployment

These three terms get used interchangeably and shouldn't be:

- **Continuous Integration (CI)** — every code change is automatically built and tested the moment it's pushed. The output is a pass/fail signal, not a deployment.
- **Continuous Delivery (CD)** — every change that passes CI is automatically packaged into a release that's *ready* to deploy, with the actual production deployment triggered by a human (a button click).
- **Continuous Deployment** — every change that passes CI is automatically deployed to production with **no human in the loop at all**.

Why the distinction matters. "We do CI/CD" is often used to mean "we have a pipeline," which tells you nothing about how much human judgment sits between a merged PR and production traffic. Etsy (Part IX) and Netflix run continuous deployment; many regulated industries (banking, healthcare) deliberately stop at continuous delivery because a compliance sign-off is a hard requirement, not a process gap to eliminate.

Building a pipeline as code

GitHub Actions — the most common entry point today, because it's colocated with the repo:


```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm test
      - run: npm run lint

  build-and-push:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ghcr.io/myorg/myapp:${ github.sha }
```

`${{ github.sha }}` as the image tag, not `latest`. Tagging every build `latest` means you can never answer “which exact code is running in production right now” without reading logs from the deploy itself. Tagging with the commit SHA makes every deployed artifact traceable back to an exact, immutable commit — the single most useful habit for debugging “it worked yesterday” incidents.

GitLab CI — the same shape, expressed with GitLab’s native syntax and built-in stages:

```
# .gitlab-ci.yml
stages: [test, build, deploy]

test:
  stage: test
  image: node:20
  script:
    - npm ci
    - npm test

build:
  stage: build
  image: docker:24
  services: [docker:24-dind]
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
  only: [main]

deploy:
  stage: deploy
  script:
    - kubectl set image deployment/myapp
      myapp=$CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
  environment: production
  only: [main]
  when: manual # this line is the difference between continuous delivery
                  and continuous deployment
```

Jenkins — older, more infrastructure to self-host, but still dominant in large enterprises with complex, custom pipeline requirements:

```
// Jenkinsfile
pipeline {
  agent any
  stages {
    stage('Test') {
      steps { sh 'npm ci && npm test' }
    }
    stage('Build') {
      steps { sh 'docker build -t myapp:${GIT_COMMIT} .' }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps { sh 'kubectl set image deployment/myapp
myapp=myapp:${GIT_COMMIT}' }
    }
  }
  post {
    failure {
      slackSend(message: "Build failed: ${env.BUILD_URL}")
    }
  }
}
```

The common thread across all three. A pipeline is just: run on a trigger → run tests → build an artifact → conditionally deploy it. The syntax differs; the shape (test gate → build → deploy gate) is identical everywhere, which is why moving between GitHub Actions, GitLab CI, and Jenkins is mostly a syntax problem, not a conceptual one, once this shape is internalized.

Artifact management & versioning

A build produces something that needs a durable, versioned home before it's deployed — this is what artifact registries are for, and skipping this step (deploying straight from a build machine) is a recurring root cause of "it worked on the build server but not in production."

Artifact type	Registry
Container images	Docker Hub, GitHub Container Registry (GHCR), Amazon ECR, Google Artifact Registry
npm packages	npm registry, GitHub Packages, a private registry (Verdaccio)
Java artifacts (JAR/WAR)	Nexus, JFrog Artifactory
Generic binaries	Artifactory, S3 with versioning enabled

Semantic versioning (SemVer) — `MAJOR.MINOR.PATCH` — is the convention that makes automated dependency updates and rollbacks safe to reason about: a `PATCH` bump (`1.2.3` → `1.2.4`) should never break a consumer; a `MINOR` bump adds functionality without breaking anything; a `MAJOR` bump is allowed to break things. Automated tools (Dependabot, Renovate) rely on this convention actually being followed to decide whether an update is safe to auto-merge.

Part III — Containers & Orchestration

Docker, deep dive

A container is not a lightweight VM — there's no separate kernel. A container is a Linux process with restricted, isolated views of the filesystem, network, and process tree, using kernel features (namespaces for isolation, cgroups for resource limits) that existed before Docker; Docker's contribution was packaging those primitives behind a usable image format and CLI.

Images are built from layers, and each layer is only the *diff* from the layer below it — this is why identical base-image lines across many images share disk space and why layer *order* in a Dockerfile has a real performance impact:

```
FROM node:20-slim

WORKDIR /app

# Copy dependency manifests FIRST, install, THEN copy the rest of the
# source.
# Docker caches each layer; if package.json hasn't changed, this layer is
# reused
# and `npm ci` doesn't re-run just because you edited an unrelated source
# file.
COPY package.json package-lock.json ./
RUN npm ci --production

COPY . .

USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

Mistake. `COPY . .` before `RUN npm ci`. Every single code change — even a one-line typo fix — invalidates the dependency-install layer's cache, so every build reinstalls every dependency from scratch. Ordering `COPY package*.json` + install *before* copying the rest of the source is the single most common Docker build-speed fix.

Multi-stage builds — the standard way to keep a production image small by not shipping build tools (compilers, full `devDependencies`) into the final image:

```
# Stage 1 – has the full toolchain, produces a compiled bundle
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2 – a minimal runtime image; only the compiled output is copied over
FROM node:20-slim
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
CMD ["node", "dist/server.js"]
```

.dockerignore — the Dockerfile equivalent of **.gitignore**, and skipping it is a real security and performance bug, not just clutter:

```
node_modules
.git
.env
*.log
```

Why this matters. Without excluding **.env**, a **COPY . .** line bakes production secrets directly into an image layer — one that persists in the image history even if a later layer deletes the file, and one that ships to every registry and every machine that pulls the image. This is a real, common way secrets leak.

Networking and volumes — containers are ephemeral and isolated by default; two flags fix the two most common “why can’t my container talk to X / why did my data disappear” questions:

```
# Containers on the same user-defined network can reach each other by
  container name

docker network create app-net
docker run -d --name db --network app-net postgres:16
docker run -d --name api --network app-net -e DB_HOST=db myapi

# Volumes persist data outside the container's writable layer –
# without one, `docker rm` on a database container deletes its data
  permanently

docker run -d --name db -v pgdata:/var/lib/postgresql/data postgres:16
```

Kubernetes, deep dive

Docker runs one container. Kubernetes answers the next question: **what runs these containers across hundreds of machines, restarts them when they crash, and routes traffic to whichever ones are healthy right now?**

Architecture — a cluster has a control plane (the brain) and worker nodes (where containers actually run):

Component	Role
API Server	The single entry point — every <code>kubectl</code> command and every internal component talks to the cluster only through this
etcd	The cluster's entire state — every object's desired and current state lives here; losing <code>etcd</code> without a backup means losing the cluster's memory of what it's supposed to be running
Scheduler	Decides which node a new Pod runs on, based on resource requests and constraints
Controller Manager	Constantly reconciles: "the Deployment says 3 replicas should exist, only 2 do — create one"
Kubelet (per node)	The agent that actually starts/stops containers on that node and reports their status back

The core objects, in the order you actually use them:

```

# A Deployment – declares "I want 3 replicas of this Pod spec, always"
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels: { app: myapp }
  template:
    metadata:
      labels: { app: myapp }
    spec:
      containers:
        - name: myapp
          image: ghcr.io/myorg/myapp:a1b2c3d
          ports: [{ containerPort: 3000 }]
          resources:
            requests: { cpu: "100m", memory: "128Mi" }  # guaranteed minimum
            limits: { cpu: "500m", memory: "256Mi" }  # hard ceiling
          readinessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 5
          livenessProbe:
            httpGet: { path: /health, port: 3000 }
            periodSeconds: 10
---
# A Service – a stable network identity in front of Pods that come and go
apiVersion: v1
kind: Service
metadata:
  name: myapp-svc
spec:
  selector: { app: myapp }
  ports: [{ port: 80, targetPort: 3000 }]
---
# An Ingress – routes external HTTP traffic into Services based on host/path
apiVersion: networking.k8s.io/v1

```



```
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service: { name: myapp-svc, port: { number: 80 } }
```

readinessProbe vs. livenessProbe — the distinction that causes real outages

when confused. A failed **readiness** probe removes the Pod from the Service's routing (no traffic sent to it) without killing it — correct for "I'm temporarily busy/warming up." A failed **liveness** probe kills and restarts the container — correct for "I'm deadlocked and will never recover on my own." Using a liveness probe for a slow-starting app causes Kubernetes to kill and restart a Pod that just needed more time, often in an infinite restart loop (`CrashLoopBackOff`) that never actually finishes starting.

Resource requests vs. limits — `requests` is what the scheduler uses to decide which node has room for this Pod; `limits` is the hard ceiling enforced at runtime. A container that exceeds its memory `limit` is OOM-killed; a container with no `limits` set at all can starve every other Pod on the same node — this is one of the most common causes of "unrelated services degraded at the same time" incidents in a shared cluster.

ConfigMaps and Secrets — externalizing configuration from the image itself, so the same image can be promoted from staging to production unchanged:

```

apiVersion: v1
kind: ConfigMap
metadata: { name: myapp-config }
data:
  LOG_LEVEL: "info"
---
apiVersion: v1
kind: Secret
metadata: { name: myapp-secret }
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQxMjM= # base64 – NOT encryption; see the
                                DevSecOps section

```

Mistake. Treating a Kubernetes `Secret` as sufficiently secure by itself. Base64 is an encoding, not encryption — anyone with `get secret` RBAC permission (or access to an unencrypted `etcd` backup) can trivially decode it. Production clusters should enable encryption at rest for `etcd` and/or use an external secrets manager (Vault, AWS Secrets Manager) with a Kubernetes integration, covered in Part VII.

Horizontal Pod Autoscaling — scaling replica count based on observed load instead of a fixed number:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: myapp-hpa }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: myapp }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }

```

Helm: templating Kubernetes manifests

Real applications need dozens of YAML files (Deployment, Service, Ingress, ConfigMap, HPA — per environment), and hand-copying them for staging vs. production is exactly the kind of repetitive manual work Part I's Automation principle says should become tooling. Helm is a templating engine plus a packaging format ("charts") for exactly this:

```
# values.yaml – the knobs that differ per environment
replicaCount: 3
image:
  repository: ghcr.io/myorg/myapp
  tag: a1b2c3d
resources:
  requests: { cpu: 100m, memory: 128Mi }
```

```
# templates/deployment.yaml – the template, filled in from values.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-myapp
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: myapp
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          resources: {{ toYaml .Values.resources | nindent 12 }}
```

```
helm install myapp ./mychart -f values-production.yaml
helm upgrade myapp ./mychart -f values-production.yaml --set
  image.tag=d4e5f6a
helm rollback myapp 1  # instant rollback to a previous release revision
```

GitOps: the cluster as a git repo

GitOps takes the “small batch, everything reviewed, everything traceable” principle from Part I and applies it to infrastructure itself: **the desired state of the cluster lives as YAML in a git repo, and**

an in-cluster controller (ArgoCD or Flux) continuously reconciles the live cluster to match that repo — nobody runs `kubectl apply` by hand against production.

```
# An ArgoCD Application – "keep this cluster in sync with this repo path,
always"
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: myapp }
spec:
  source:
    repoURL: https://github.com/myorg/k8s-manifests
    path: apps/myapp/production
    targetRevision: main
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      selfHeal: true  # if someone manually kubectl-edits the cluster,
                      revert it back to match git
```

Why this matters. `selfHeal: true` is the concrete enforcement of “git is the single source of truth” — a manual, undocumented `kubectl edit` against production (a very common way configuration drifts silently) gets automatically reverted, and any real change has to go through a PR, which means it’s reviewed and has a permanent audit trail. This directly addresses the kind of undocumented-manual-change root cause that shows up repeatedly in Part IX.

Part IV — Infrastructure as Code & Config Management

Terraform, deep dive

Before Terraform, provisioning a server meant clicking through a cloud console — unrepeatable, undocumented, and impossible to review in a PR. **Infrastructure as Code (IaC)** treats infrastructure definitions the same way as application code: version-controlled, reviewed, and applied through a pipeline.

```
# main.tf
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {                                # remote state – see below
    bucket = "myorg-terraform-state"
    key     = "prod/network.tfstate"
    region  = "us-east-1"
    dynamodb_table = "terraform-locks"    # state locking – see below
  }
}

provider "aws" { region = "us-east-1" }

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags = { Name = "production-vpc" }
}

resource "aws_instance" "web" {
  count          = 3
  ami            = "ami-0c55b159cbfafa1f0"
  instance_type = "t3.medium"
  subnet_id     = aws_subnet.public.id
  tags = { Name = "web-${count.index}" }
}
```

```
terraform init      # download providers, configure the backend
terraform plan      # show what WOULD change – the single most important
                    # safety habit
terraform apply     # actually make the change
```

terraform plan before every **apply** is non-negotiable in production. **plan** is a dry run that shows exactly what will be created, changed, or — critically — **destroyed**, before anything happens. Skipping straight to **apply** (or auto-approving in CI without a human or automated policy check reading the plan output) is how a small, intended change turns into an unintended full resource replacement, because certain attribute

changes (like renaming a resource) force Terraform to destroy and recreate rather than update in place.

State: what it is and why remote + locked state matters. Terraform's `.tfstate` file is its record of what it believes currently exists and maps it to your `.tf` definitions. Two people running `terraform apply` against the *same local state file* at the same time can corrupt it or silently overwrite each other's changes — which is exactly why production Terraform always uses a **remote backend** (S3, Terraform Cloud) with **state locking** (the `dynamodb_table` line above): the lock means a second `apply` has to wait for the first to finish instead of racing it.

Modules — the way Terraform avoids copy-pasted resource blocks across environments:

```
module "vpc" {  
  source    = "../modules/vpc"  
  cidr      = "10.0.0.0/16"  
  env_name  = "production"  
}  
  
module "vpc_staging" {  
  source    = "../modules/vpc"  
  cidr      = "10.1.0.0/16"  
  env_name  = "staging"  
}
```

Drift — the state of the world silently diverging from what Terraform's state file believes is true, usually from someone making a manual console change. `terraform plan` surfaces drift by comparing real infrastructure against state before comparing state against `.tf` files — running `plan` regularly (or in a scheduled CI job) even with no intended change is a legitimate drift-detection practice.

Ansible: configuration management

Terraform answers “what infrastructure exists” (a VM, a network). Ansible answers “what's installed and configured *on* that infrastructure” (packages, config files, running services) — the two are complementary, and a common real pipeline runs Terraform first to provision, then Ansible to configure.

```
# playbook.yml
- hosts: web_servers
  become: true
  tasks:
    - name: Install nginx
      apt: { name: nginx, state: present, update_cache: true }

    - name: Deploy nginx config
      template:
        src: nginx.conf.j2
        dest: /etc/nginx/nginx.conf
      notify: restart nginx    # only restarts if this task actually changed the file

    - name: Ensure nginx is running
      service: { name: nginx, state: started, enabled: true }

  handlers:
    - name: restart nginx
      service: { name: nginx, state: restarted }
```

```
ansible-playbook -i inventory.ini playbook.yml
```

Idempotency is the entire point. Running the same Ansible playbook 1 time or 100 times against the same server should produce the exact same end state, and running it a second time with nothing to do should report “0 changed” rather than re-running every task blindly. This is what makes it safe to re-run a playbook after a partial failure instead of needing to manually figure out what already happened — the opposite of a bash script that blindly re-installs and re-appends config on every run.

Terraform vs. Ansible vs. Pulumi: when to use which

Tool	Best for	Model
Terraform	Provisioning cloud infrastructure (VPCs, VMs, managed databases, Kubernetes clusters themselves)	Declarative HCL, explicit state file
Ansible	Configuring what runs on machines that already exist (packages, files, services) — also fine for one-off orchestration tasks	Declarative YAML, agentless (SSH), no persistent state file
Pulumi	Same job as Terraform, for teams that want infrastructure defined in a real programming language (TypeScript, Python, Go) instead of HCL	Declarative, but authored imperatively — loops/conditionals are native language features, not HCL workarounds

Part V — Cloud Fundamentals

AWS core services for DevOps

Every cloud provider maps onto the same conceptual categories; AWS is used here as the concrete reference since it remains the most commonly required in job postings.

Category	Service	What it's for
Compute	EC2	Virtual machines you fully manage
Compute	ECS / EKS	Managed container orchestration (ECS is AWS-proprietary; EKS is managed Kubernetes)
Compute	Lambda	Run code per-request with no server to manage — pay per invocation
Networking	VPC	An isolated virtual network — the foundation everything else sits inside
Networking	Route 53	DNS
Networking	ELB/ALB	Load balancing across multiple instances/containers
Storage	S3	Object storage — the backbone of static assets, backups, and Terraform remote state
Storage	EBS	Block storage attached to a single EC2 instance
Identity	IAM	Who (or what service) can do what — the single most security-critical service in the account
Database	RDS	Managed relational databases (Postgres, MySQL)

IAM's least-privilege principle, in practice — the difference between a policy that “works” and one that's actually safe:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::myapp-uploads/*"
  }]
}
```

Mistake. Granting `"Action": "s3:*"` on `"Resource": "*"` because it's faster to get something working. An over-broad IAM policy attached to an application's role means a single application-layer vulnerability (an SSRF, a dependency compromise) can pivot into full account compromise instead of being contained to the one bucket that application actually needs. Scoping IAM policies to exact actions and exact resource ARNs is one of the highest-leverage, lowest-cost security practices available — see Part VII.

Part VI — Observability & SRE

The three pillars: metrics, logs, traces

"Monitoring" and "observability" aren't quite the same thing: monitoring tells you *that* something is wrong (a dashboard, an alert); observability is having enough data of the right shape that you can answer *any* question about *why*, including ones you didn't think to ask in advance. That requires all three pillars, because each answers a different question:

Pillar	Answers	Tooling
Metrics	"How much / how many, over time?" (request rate, error rate, latency, CPU)	Prometheus, Datadog, CloudWatch
Logs	"What exactly happened, in this specific request/event?"	ELK/EFK stack, Loki, CloudWatch Logs
Traces	"Where did the time go, across every service this one request touched?"	Jaeger, Zipkin, OpenTelemetry

Prometheus + Grafana

Prometheus **pulls** metrics — it scrapes an HTTP endpoint (`/metrics`) that each service exposes, on an interval, rather than services pushing data to it. This pull model makes it trivial to see if a target is down at all (a failed scrape is itself a signal) without needing every service to reliably push.

```
# prometheus.yml
scrape_configs:
  - job_name: 'myapp'
    scrape_interval: 15s
    static_configs:
      - targets: ['myapp:3000']
```

```
# PromQL – the query language, a few examples
rate(http_requests_total[5m])           # requests/sec over the last 5
minutes
histogram_quantile(0.99, http_request_duration_seconds_bucket) # p99
latency
rate(http_requests_total{status=~"5.."}[5m]) # error rate, 5xx only
```

```
# Alertmanager rule – page someone when error rate crosses a threshold
groups:
  - name: myapp
    rules:
      - alert: HighErrorRate
        expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
        for: 10m           # must stay true for 10 minutes – avoids paging
                           on a single blip
        annotations:
          summary: "Error rate above 5% for 10 minutes"
```

Grafana sits on top as the visualization and dashboard layer, querying Prometheus (and other data sources) to render the graphs a team actually watches during an incident.

Mistake. Alerting on raw instantaneous values instead of a rate sustained **for** a duration. A single scrape catching a one-second blip pages someone for a non-issue; the **for:** **10m** clause is what separates “real, sustained degradation” from noise — alert fatigue from

noisy thresholds is one of the most common reasons real alerts get ignored during an actual incident.

Centralized logging: the ELK/EFK stack

With more than a couple of servers, `ssh`-ing in to `tail -f` a log file stops working — logs need to be shipped somewhere centralized and searchable. **ELK** = Elasticsearch (storage + search) + Logstash (ingestion/parsing) + Kibana (visualization); **EFK** swaps Logstash for Fluentd/Fluent Bit, a lighter-weight log shipper more common in Kubernetes.

```
# Fluent Bit – runs as a DaemonSet, ships every container's stdout to
Elasticsearch

[INPUT]
  Name tail
  Path /var/log/containers/*.log

[OUTPUT]
  Name es
  Match *
  Host elasticsearch.logging.svc
  Port 9200
```

Why this matters. In Kubernetes specifically, a Pod's filesystem (and its logs) disappear the instant it's rescheduled or crashes — which is exactly when you most need the logs. Centralized logging that ships logs out of the container as they're written is the only way to debug a Pod that's already gone by the time you look.

Distributed tracing: OpenTelemetry + Jaeger

A single user request in a microservices architecture might touch a dozen services. Metrics tell you the *overall* p99 latency went up; a trace tells you *which one* of those twelve services actually accounted for the extra time.

```
// OpenTelemetry – instrumenting one span in a Node.js service
const span = tracer.startSpan('fetch-user-orders');
try {
  const orders = await ordersService.getOrders(userId);
  span.setStatus({ code: SpanStatusCode.OK });
  return orders;
} finally {
  span.end();
}
```

Each span carries a trace ID that propagates across service-to-service calls (via HTTP headers), so a tool like Jaeger can reassemble the full request path as a single waterfall diagram — this is what turns “the checkout API is slow” into “the checkout API is slow because the inventory service it calls is slow, specifically on its database query.”

SRE concepts: SLIs, SLOs, error budgets, postmortems

Site Reliability Engineering (the discipline Google popularized) gives DevOps culture a quantitative way to decide “are we reliable enough, and how much risk can we afford to take this quarter”:

- **SLI (Service Level Indicator)** — an actual measured metric: “percentage of requests served in under 300ms,” “percentage of successful requests.”
- **SLO (Service Level Objective)** — the internal target for that SLI: “99.9% of requests succeed in under 300ms, measured over 30 days.”
- **SLA (Service Level Agreement)** — the SLO, but a contractual promise to a customer, usually with a financial penalty for missing it. Every SLA is built on an SLO; not every SLO is a public SLA.
- **Error budget** — the inverse of the SLO: if the SLO is 99.9%, the error budget is the 0.1% of allowed failure. This reframes reliability as a *resource to spend*, not an absolute requirement: as long as the error budget isn’t exhausted, teams are explicitly cleared to take risks (ship faster, run experiments); once it’s exhausted, the team’s priority shifts to reliability work until the budget recovers.

Why this matters — it resolves the dev-vs-ops tension from Part I with a number instead of an argument. Without an error budget, “can we ship this risky change” is a political negotiation between a dev team that wants velocity and an ops team that wants stability. With an error budget, it’s a data-driven answer: there’s budget left, so ship it; the budget’s exhausted, so this sprint is about reliability, not features — both sides are arguing from the same shared number instead of from opposing incentives.

Blameless postmortems — the standard practice after any significant incident, and the one most directly visible in Part IX's GitLab case study: the goal is documenting *what happened and what systemic gap allowed it*, explicitly not *whose fault it was*. The reasoning is pragmatic, not just kind: an engineer who fears being blamed for an incident will hide the details of what actually happened next time, and the organization loses the only chance to fix the systemic issue rather than just punishing the last person who tripped over it.

Part VII — Security

DevSecOps: shifting security left

"Shift left" means moving security checks earlier in the pipeline (into the PR, before merge) instead of only running them right before — or after — a production release, when a finding is far more expensive to fix and far more likely to already be exploited.

Check	What it catches	Where it runs
SAST (Static Application Security Testing)	Vulnerable code patterns (SQL injection, hardcoded secrets) by analyzing source code without running it	In CI, on every PR
SCA (Software Composition Analysis)	Known CVEs in your dependencies (<code>npm audit</code> , Snyk, Dependabot)	In CI, and continuously against what's already deployed
DAST (Dynamic Application Security Testing)	Vulnerabilities found by actually attacking a running instance of the app (OWASP ZAP)	Against a staging environment, pre-release
Image scanning	Known CVEs in OS packages baked into a container image (Trivy, Gype)	In CI, before an image is pushed to a registry

```
# A CI step that fails the build on a critical vulnerability, not just a warning
- name: Scan image
  run: trivy image --severity CRITICAL --exit-code 1 ghcr.io/myorg/myapp:${{github.sha }}
```

Secrets management — the discipline of never letting a credential live in source code, a Docker image layer, or plain-text Kubernetes YAML at all:

```
# Bad – a secret committed to git is compromised forever, even after
      deletion,
# because it exists in git history and possibly in every fork/clone
DB_PASSWORD=hunter2 node server.js

# Good – fetched at runtime from a dedicated secrets manager, never touching
      disk or git
vault kv get -field=password secret/myapp/db
```

HashiCorp Vault (or a cloud-native equivalent — AWS Secrets Manager, GCP Secret Manager) centralizes secret storage, supports automatic rotation, and provides an audit log of exactly which service accessed which secret and when — none of which a `.env` file or a base64 Kubernetes Secret can give you.

The recurring root cause worth naming explicitly. A large fraction of real-world breaches trace back not to a novel zero-day exploit, but to a leaked credential — an API key committed to a public repo, an over-permissioned IAM role, a secret baked into a Docker image layer (the exact mistake flagged in Part III's `.dockerignore` section). Shift-left security tooling exists specifically to catch these boring, common mistakes automatically, before a human reviewer has to remember to check for them by eye.

Part VIII — Deployment Strategies & Resilience

Blue-green, canary, and rolling deployments

All three exist to answer the same question — how do you replace a running production version with a new one without downtime and without exposing every user to a broken release at once — with different risk/cost trade-offs:

Strategy	How it works	Trade-off
Rolling deployment	Replace old-version instances with new-version instances a few at a time (Kubernetes's default Deployment behavior)	Simple, no extra infrastructure cost, but a bad release is already serving some real traffic before it's noticed
Blue-green	Run the full new version ("green") alongside the full old version ("blue"), then switch all traffic over at once (a load balancer or DNS change)	Instant rollback (switch back to blue), but doubles infrastructure cost during the switch
Canary	Route a small percentage of real traffic (e.g. 5%) to the new version, watch its metrics, then gradually increase	Catches a bad release with minimal blast radius, but needs good enough observability (Part VI) to actually judge "is the canary healthy" automatically

```
# A canary step in Argo Rollouts (a Kubernetes-native progressive delivery controller)
```

```
strategy:
  canary:
    steps:
      - setWeight: 5      # 5% of traffic to the new version
      - pause: { duration: 10m }
      - setWeight: 25
      - pause: { duration: 10m }
      - setWeight: 100
```

Chaos engineering

Chaos engineering is the practice of **deliberately injecting failure into a production (or production-like) system to verify it actually survives failures it's supposed to be designed for**, rather than assuming resilience because the design doc says so. Part IX covers Netflix's Chaos Monkey as the origin case in detail.


```
# A conceptual example – a chaos experiment definition
```

```
experiment:
```

```
  hypothesis: "Killing one payment-service pod causes zero customer-visible errors"
```

```
  action: kill-random-pod
```

```
  target: { label: app=payment-service }
```

```
  steady_state_check: error_rate < 0.1%
```

Why this matters. A system diagram showing “3 replicas, load balanced” *looks* resilient, but that’s a claim about design intent, not a verified fact about the running system — the only way to know the failover, health check, and retry logic actually work together correctly under real failure is to actually trigger the failure and watch. Teams that only discover a broken failover path during a real incident are discovering it at the worst possible time; chaos engineering moves that discovery to a planned, low-stakes moment.

Disaster recovery: RTO, RPO, and backups

- **RTO (Recovery Time Objective)** — how long can the system be down before the business impact is unacceptable? Drives *how much automation and standby infrastructure* you need.
- **RPO (Recovery Point Objective)** — how much data can you afford to lose, measured in time? Drives *how frequently you back up / replicate*.

A backup that’s never been tested with an actual restore is not a backup — it’s an unverified assumption. GitLab’s 2017 incident (Part IX) is the canonical case study for exactly this gap.

Part IX — Real-World Incidents

Knight Capital: \$440 million in 45 minutes

What happened. In August 2012, Knight Capital, a major market-making firm, deployed new trading software to its production servers. The deployment process required manually copying new code to eight production servers — a human missed one. That eighth server kept running old, dormant code that included a disabled feature flag repurposed years earlier; the new deployment reused that same flag for a different feature. When markets opened, the eighth server activated the old, dormant code path, which began sending a flood of unintended, erroneous orders into the market.

Root cause. A **manual deployment process with no automated verification that all servers were actually running the same version** — exactly the kind of repetitive manual step Part I's Automation principle calls out as needing to become tooling. There was no automated check comparing the deployed version across all eight servers, and no kill switch that could halt trading the moment order flow looked abnormal.

How they “solved” it (the industry's response, since Knight Capital itself lost \$440 million in 45 minutes and was acquired shortly after): the incident became a standard case study driving the adoption of **automated, immutable deployments** (deploy the *exact same* verified artifact to every server, with no server-specific manual steps — see Part III's container image approach, where an image tagged with a commit SHA is bit-for-bit identical everywhere it runs) and **automated canary/circuit-breaker checks** that can halt a rollout the instant a key metric (like order volume) deviates from expectations, rather than relying on a human to notice in time.

The lesson that maps directly onto this guide. Every practice in Parts II–III — pipeline-as-code (no manual deploy steps), immutable, SHA-tagged container images (no “which version is actually on server 8” ambiguity), and canary deployments with automated rollback (Part VIII) — exists specifically to make a Knight-Capital-shaped incident structurally impossible, not just less likely.

AWS S3, February 2017: a typo takes down half the internet

What happened. An AWS engineer was debugging a slow-running billing system for S3 in the US-EAST-1 region. Following an established runbook to remove a small number of servers from one S3 subsystem, they ran a command with an incorrect parameter — one that removed a much larger set of servers than intended, including servers supporting two other critical S3 subsystems. Those subsystems required a full restart, and — because they hadn't been fully restarted in years — the restart itself took far longer than expected, extending an outage that ultimately lasted about 4 hours and took down a huge swath of the internet that depended on S3 (including parts of AWS's own status dashboard, which itself relied on S3).

Root cause. A **manual command run against production with insufficient guardrails** — the tooling allowed an input that removed more capacity than any legitimate operation should ever need, with no automated lower-bound check rejecting an obviously-too-large removal request. A secondary contributing cause: the affected subsystems had never been restarted at that scale in years, so the actual restart time under real conditions was unknown until it happened live.

How AWS addressed it. AWS's public post-incident summary described adding safety checks that prevent removing capacity below a minimum required level regardless of what command is entered, and — notably — **breaking the S3 status dashboard's own dependency on the S3**

region it monitors, so that a regional S3 outage can no longer also take down the dashboard reporting that outage.

The lesson. This is a textbook case for **guardrails over trust** — the fix wasn't "tell engineers to be more careful," it was making the dangerous input structurally impossible to submit. It's also a sharp illustration of **hidden circular dependencies** (a monitoring system depending on the very thing it monitors) — worth actively auditing for in any observability setup from Part VI.

GitLab.com, 2017: the database incident and its radically public postmortem

What happened. During routine maintenance to address replication lag on a PostgreSQL database, an engineer, working to resolve replication issues under pressure, ran a command intended to clear an empty directory on a secondary replica — but ran it against the **primary** database server instead, deleting roughly 300GB of production data, including issues, merge requests, and user data for GitLab.com.

The part that made this incident famous: when the team went to restore from backups, they discovered — live, during the incident — that **none of their five separate backup/replication mechanisms had been working correctly**. Regular database dumps had been failing silently for weeks because the backup script's error output wasn't being monitored. Other replication methods were also found to be broken or misconfigured. The team ultimately restored from a snapshot that was several hours old, permanently losing that window of data.

Root cause. Two independent failures stacked together: (1) a manual, high-risk command run against the wrong target under time pressure, with no confirmation step distinguishing primary from replica, and (2) **backups that had never been verified with an actual test restore**, so their silent failure went undetected until the moment they were needed — exactly the gap named in Part VIII's Disaster Recovery section.

How GitLab responded. Rather than a private internal postmortem, GitLab **livestreamed the recovery process itself** and published a fully detailed, public incident report naming the exact commands run and exact systems that failed — an unusually extreme, and widely praised, application of the blameless-postmortem principle from Part VI. The concrete follow-up work included implementing automated, regularly-tested backup verification (a scheduled job that doesn't just run a backup, but actually attempts a restore and confirms it succeeded), and adding explicit environment indicators to prevent commands from being run against the wrong host.

The lesson. A backup you have not test-restored is a hypothesis, not a safety net. This single incident is the strongest real-world argument for the Part VIII principle that disaster

recovery capability must be periodically *exercised*, not just configured and assumed to work.

Meta, October 2021: the BGP outage that locked engineers out of their own building

What happened. Facebook, Instagram, and WhatsApp all became unreachable for roughly six hours. The root cause was a routine maintenance command intended to assess the availability of global backbone network capacity — a bug in the auditing tool that was supposed to check the command's safety failed to catch that it would disconnect Facebook's data centers from the internet entirely. This withdrew the **BGP (Border Gateway Protocol)** routes that tell the rest of the internet how to reach Facebook's servers, and — as a side effect — also caused Facebook's own DNS servers to withdraw their routes as a safety mechanism, so `facebook.com` stopped resolving at all, for anyone, anywhere.

What made this particularly severe: the outage **also broke Facebook's own internal tools** — including the badge-reader systems engineers needed to physically access the data centers to fix the problem, and internal engineering tools that themselves depended on the now-unreachable network. Engineers had to gain physical access to secure facilities to manually reset the affected systems, significantly extending the outage.

Root cause. A configuration change deployed through an automated audit/rollout system whose safety check had a bug, combined with **insufficient blast-radius isolation** — a single bad config change should not have been able to cascade into disconnecting the entire global network, and should especially not have been able to disable the physical/internal tooling needed to respond to it.

How Meta responded. Their public postmortem described work to slow down and add more staged rollout/verification for this class of backbone configuration change, and — critically — building more resilient, independent recovery tooling that doesn't depend on the same infrastructure it might need to help recover.

The lesson. This is the most extreme real-world case of a principle every Part III–IV section touches on: **the tooling used to fix a production problem must not have a hard dependency on the thing that's currently broken.** A GitOps controller, an emergency runbook, or a break-glass admin account should be reachable and functional even when the primary system they manage is completely down.

Netflix: engineering for failure on purpose

What happened (as an ongoing practice, not a single incident). As Netflix migrated to AWS in the late 2000s/early 2010s, they made a deliberate architectural bet: instead of trying to prevent every possible failure, assume failures (a server dying, a network partition, an AWS zone outage) are inevitable and continuous, and build systems — and a culture — that treat surviving them as a normal, expected, and *regularly tested* condition rather than an exceptional one.

The tooling this produced became the origin of modern chaos engineering (Part VIII): **Chaos Monkey**, which randomly terminates production instances during business hours specifically so that engineers are forced to build services that tolerate an instance dying at any moment, rather than assuming it won't happen. This expanded into the "Simian Army" — tools that simulated an entire AWS availability zone failing, unusual traffic spikes, and even simulated security compromise, all against production, on a regular schedule.

Why this counts as "solving" a problem rather than causing one. The point isn't the outage each tool causes — it's that **any weakness these tools find is found on Netflix's own schedule, during business hours, with engineers already watching**, instead of being found for the first time during an unplanned 3 AM real outage. This is the direct, deliberate application of the chaos engineering principle from Part VIII at the scale of an entire company's culture, not just a single tool.

The lesson. Resilience claimed by a system diagram is unverified until it's actually tested under real failure. Netflix's contribution to the industry wasn't a single tool — it was normalizing the idea that **you should find your own outages before your users do**, on purpose, repeatedly.

Etsy: from twice-a-week deploys to fifty times a day

What happened. In the mid-2000s, Etsy's deployment process was slow, risky, and infrequent — deploys happened on a fixed schedule, each one bundling a large batch of changes, and each one carried enough risk that engineers dreaded deploy days. Etsy's engineering leadership made a deliberate, multi-year cultural and technical investment to flip this: instead of making deploys rarer to reduce risk, they made deploys **smaller and more frequent** specifically *to reduce* risk — directly applying the Lean principle from Part I's CALMS framework (small batch size reduces the blast radius of any single change).

What they built to make this possible: - A **continuous deployment pipeline** (Part II) where a merged change could go to production within minutes, not days. - **Feature flags**, decoupling *deploying* code from *releasing* a feature to users — new code could sit dormant in production, flagged off, and be enabled gradually or instantly rolled back without a redeploy. - A famous internal practice where **every new engineer deploys to production on their first day**, specifically to normalize deployment as a routine, low-stress, well-tooled event rather than a rare, high-

ceremony one. - Deep investment in the observability (Part VI) needed to make frequent small deploys safe — if you can't quickly tell whether the last 10-minute deploy caused a problem, deploying 50 times a day is reckless, not brave.

Why this matters as a case study rather than just a fun fact. Etsy's trajectory is the practical, positive counterpoint to Knight Capital: the same underlying goal (safely change production software) produced an opposite strategy — instead of tightly controlling a rare, large deployment, radically increase deployment frequency and shrink each change until the risk of any single deploy approaches zero.

The lesson. This is the clearest real-world validation of DORA's trunk-based-development finding from Part I: high deploy frequency and low change failure rate are not in tension — done with the right tooling (CI/CD, feature flags, observability), they reinforce each other, because smaller changes are inherently easier to reason about, test, and roll back.

Part X — The Roadmap

A structured roadmap, 0–12 months

This roadmap assumes basic programming ability (any language) and no prior ops experience. Each stage builds on the last — don't skip Linux/Git to jump straight to Kubernetes; nearly every debugging skill downstream depends on it.

Months 1–2 — Foundations - Linux fundamentals: the filesystem hierarchy, permissions, processes, `systemd`, package managers - Shell scripting: variables, loops, `set -euo pipefail`, writing a real automation script end to end - Git beyond the basics: rebasing, resolving conflicts, branching strategies from Part I - Networking basics: DNS, HTTP/HTTPS, TCP vs UDP, what a load balancer actually does

Months 3–4 — CI/CD & Containers - Pick one CI system (GitHub Actions is the lowest-friction starting point) and build a real pipeline: test → build → push an image - Docker: build an image for a real app you've written, understand layers, write a multi-stage build - Push that image to a registry (GHCR or Docker Hub) with proper SHA-based tagging

Months 5–7 — Orchestration & IaC - Kubernetes: run a local cluster (kind or minikube), deploy the container from months 3–4 as a Deployment + Service + Ingress - Add health probes, resource requests/limits, a ConfigMap and Secret - Terraform: provision the actual cloud infrastructure (a VPC, a managed Kubernetes cluster) that your Pods run on, with remote state - Helm: package your Kubernetes manifests as a chart with environment-specific values files

Months 8–9 — Observability & Cloud - Deploy Prometheus + Grafana against your cluster; instrument your app to expose a `/metrics` endpoint - Set up centralized logging (EFK or a hosted equivalent) so you can debug a Pod after it's gone - Go one level deeper on one cloud provider's IAM model — practice least-privilege policies, not just “make it work” policies

Months 10–11 — Security & Resilience - Add image scanning (Trivy) and dependency scanning to your CI pipeline - Move any hardcoded secret into a real secrets manager (Vault, or your cloud's native equivalent) - Practice a deployment strategy beyond rolling — implement a canary step with Argo Rollouts or an equivalent - Read all six incidents in Part IX again with your own pipeline in mind — which of these six root causes could happen to what you just built?

Month 12 — SRE practices & synthesis - Define an actual SLO for a project you've built, and calculate its error budget - Write a blameless postmortem template and use it — deliberately break something in a test environment and practice the process - Build the capstone project (Part XI, project 10) that pulls every prior month's skill into one pipeline

Certifications worth pursuing, roughly in the order they build on each other: CKA (Certified Kubernetes Administrator) or CKAD (Application Developer) for Kubernetes depth; an AWS/Azure/GCP associate-level cloud certification for one provider; HashiCorp's Terraform Associate for IaC. These are useful as structured study guides and resume signals, not as a substitute for having actually built and broken things yourself.

Part XI — Build

Projects to build, in order

01. A CI pipeline for a real app you already have Add GitHub Actions running tests on every PR, then building and pushing a Docker image tagged with the commit SHA on merge to `main`. *Skills: Part II — pipeline as code, artifact tagging.*

02. A multi-stage Dockerized service Take that same app, write a proper multi-stage Dockerfile, add a `.dockerignore`, and verify the final image doesn't contain build tools or secrets. *Skills: Part III — Docker layers, multi-stage builds, image hygiene.*

03. A local Kubernetes deployment Run the image on a local cluster (kind/minikube) as a Deployment with a Service and Ingress, with correctly distinguished readiness/liveness probes and resource requests/limits. *Skills: Part III — Kubernetes core objects.*

04. Infrastructure as code for the cluster it runs on Provision an actual cloud-managed Kubernetes cluster (EKS/GKE/AKS) and its VPC with Terraform, using a remote, locked state backend. *Skills: Part IV — Terraform, state, modules.*

05. A Helm chart with per-environment values Package project 03's manifests as a Helm chart with `values-staging.yaml` and `values-production.yaml`. *Skills: Part III — Helm templating.*

06. GitOps-managed deployment Point ArgoCD or Flux at a git repo containing that Helm chart, with `selfHeal` enabled, and verify that a manual `kubectl edit` against the cluster gets automatically reverted. *Skills: Part III — GitOps.*

07. A full observability stack Deploy Prometheus + Grafana + an EFK/Loki logging stack against the cluster; instrument the app with a `/metrics` endpoint and structured logs; build one dashboard and one alert rule with a sensible `for` duration. *Skills: Part VI — metrics, logs, alerting.*

08. A security-hardened pipeline Add Trivy image scanning and dependency scanning (`npm audit` /Snyk/Dependabot) to the CI pipeline from project 01, failing the build on critical findings; move any secret out of plaintext config and into Vault or a cloud secrets manager. *Skills: Part VII — DevSecOps.*

09. A canary deployment with automated rollback Implement a canary rollout (Argo Rollouts or an equivalent) that shifts traffic gradually and automatically rolls back if the error-rate metric from project 07 crosses a threshold during the rollout. *Skills: Part VIII — progressive delivery, tying observability to deployment safety.*

10. Capstone: a chaos-tested, SLO-backed production pipeline Combine everything: define an SLO for the app, run a chaos experiment (kill a random Pod, simulate a dependency failure) and verify the SLO holds, then write a blameless postmortem for whatever you find broken — because something will be. *Skills: every prior part, plus Part IX's incidents and Part VI's SRE practices, applied together.*

This guide condenses practices and incidents that are individually documented in far more depth elsewhere — AWS's and GitLab's own public post-incident reports, Meta's engineering blog on the 2021 outage, Netflix's engineering blog on chaos engineering, and Etsy's own widely-cited engineering culture writeups — worth reading in full once this guide's shape is familiar.